

大模型赋能的软件测试案例研究

—— 以 SymPy simplify 模块为例

学号: [SC2616041] 姓名: [杨余松]

课程: 智能化软件开发与测试技术 日期: 2026 年 5 月

概述

软件测试是保障软件质量的核心工程活动，但人工编写高质量测试用例成本高、效率低、容易遗漏边界条件。本文以开源数学库 SymPy (GitHub 12k+ stars) 的 `simplify` 模块为真实软件对象，设计并实现了一套三阶段大语言模型 (LLM) 驱动测试生成流水线——初始生成、覆盖率缺口补充、变异杀死测试——并与人工编写的基线测试进行定量对比。实验结果表明：LLM Round 1 生成的测试在 **代码覆盖率 (39.4%) 上超过了人工基线 (37.8%)**，且每个 LLM 测试的平均覆盖率贡献 (2.38%) 是人工测试 (0.74%) 的 **3.2 倍**。同时，我们系统分析了 LLM 幻觉的 6 种典型失败模式，并设计了编译验证、导入修复、prompt 约束等兜底机制。本研究表明，LLM 可作为软件测试工程师的“副驾驶”，显著降低测试编写成本，但人工审核在语义正确性把关中仍然不可或缺。

关键词: 大语言模型; 软件测试; SymPy; 代码覆盖率; 变异测试; 幻觉分析

1. 问题背景与研究动机

1.1 软件测试的工程挑战

单元测试是保障代码质量的基石。一个理想的测试套件应当覆盖函数的正常路径、边界条件、异常输入和回归场景。然而，在实际工程中，编写高质量测试面临以下痛点：

- 认知负担高：**测试工程师需要深入理解被测代码的内部逻辑，构建覆盖各种分支的输入。
- 容易遗漏：**边界条件和罕见路径容易被忽视（例如 `n=0` 时的除零行为）。
- 维护成本大：**当源码演化时，测试用例需要同步更新。

1.2 为什么选择 SymPy

[SymPy](#) 是一个纯 Python 实现的计算机代数系统 (CAS)，在 GitHub 拥有超过 12,000 个 star，广泛应用于数学建模、物理计算和工程仿真。其 `simplify` 模块 (`sympy/simplify/simplify.py`, 约 1000 行) 是核心模块之一，负责表达式化简——这是一个“非良定义” (not well-defined) 的问题：化简策略因上下文而异，涉及多项式展开、三角恒等式、对数合并、反函数组合等十余种子策略的协同调度。

SymPy 的 `simplify` 函数是典型的“重逻辑、多分支”代码，其测试难度体现在：

特征	测试挑战
约 1000 行 Python 代码	覆盖所有路径需要大量测试用例
10+ 种化简子策略	每种策略有独立的边界条件
符号计算语义	结果等价性难以用简单 <code>==</code> 验证
参数敏感 (ratio、measure、inverse 等)	参数组合爆炸

1.3 研究问题

本文围绕以下四个问题展开：

- **Q1 (问题定义)**：SymPy simplify 模块的测试痛点是什么？LLM 能改进什么？
- **Q2 (方法设计)**：LLM 如何介入测试流程？prompt、workflow 和人工参与点如何设计？
- **Q3 (有效性证据)**：LLM 生成的测试相比人工基线，在覆盖率和变异分数上是否有可观测的改进？
- **Q4 (失败分析)**：LLM 在什么情况下会失败？我们采取了什么兜底机制？

2. 软件对象与工程场景

2.1 目标模块

本次实验聚焦 SymPy `simplify` 模块中的 **8 个公开函数**：

函数	功能	复杂度
<code>simplify</code>	通用表达式化简（核心）	高（~400 行）
<code>separatevars</code>	多变量表达式分离	中
<code>signsimp</code>	符号规范化处理	中
<code>logcombine</code>	对数合并	中
<code>posify</code>	符号正定性假设替换	低
<code>hypersimp</code>	超几何项化简	低
<code>inversecombine</code>	反函数组合化简	低
<code>nthroot</code>	n 次方根数值计算	中

2.2 工程场景：测试与质量

本文选择的工程环节是 **测试与质量 (Testing & Quality)**，具体聚焦于 **单元测试补全 (Test Augmentation)**。核心主张是：

不是展示"模型能生成测试"，而是展示"模型怎样改变了测试工程师的工作方式"。

在传统流程中，测试工程师需手动阅读源码 → 分析逻辑 → 设计用例 → 编写代码。LLM 赋能的流程将这一过程转变为：LLM 生成初稿 → 自动验证（编译、运行、覆盖率）→ 人工审核修正。

3. 方法设计

3.1 三阶段 LLM 测试生成流水线

我们设计了一套三阶段流水线，模拟测试工程师的渐进式思维过程：

LLM 赋能测试流水线		
Round 1	Round 2	Round 3
初始生成	覆盖率缺口补充	变异杀死测试
输入： 函数源码	输入： 未覆盖行号 + 已有测试	输入： 变异描述 + 源码 + 已有测试
输出： 初始测试用例	输出： 补充测试用例	输出： 杀死变异体的测试
类比： 测试工程师拿到 需求后写初稿	类比： 测试工程师看到 覆盖率报告后补测	类比： 测试工程师针对 已知缺陷写回归测试

3.2 Prompt 设计

Round 1 Prompt (prompts/initial_generation.txt)

核心设计逻辑：

- 角色设定: "资深的测试工程师，擅长使用 pytest 为 SymPy 数学库编写单元测试"
- 分步引导: 分析 → 规划 → 生成，强制 LLM 先思考再编码
- 场景覆盖约束: 明确要求至少 5 种场景（正常、边界、异常、文档示例等）
- 输出约束（经过实践迭代强化）：
 - 只输出 Python 代码，不要输出分析推理文字
 - 禁止非 ASCII 字符
 - docstring 必须正确闭合
 - 每个函数体不超过 30 行
- 提示词具体内容：

你是一位资深的测试工程师，擅长使用 `pytest` 为 `SymPy` 数学库编写单元测试。

下面是 `SymPy` 中一个函数的代码：

{code}

函数名：{function_name}

请按照以下步骤生成测试用例：

1. 分析：分析函数的输入、输出、核心算法逻辑以及可能的边界条件（如零、无穷、负数、特殊值等）。注意 `SymPy` 的符号计算特性。
2. 规划：列出至少 5 个不同的测试场景，包括：
 - 正常输入（典型用例）
 - 边界输入（极值、零、空）
 - 异常输入（类型错误、无意义表达式）
 - 与文档字符串中示例的对比（如果有）
3. 生成：为每个场景编写一个独立的测试函数，函数名以 `test_` 开头。使用 `assert` 或 `pytest.raises`。

严格要求（必须遵守）：

- 只输出 `Python` 代码，不要输出任何分析、推理、解释或自然语言。
- 不要使用中文、日文等非 `ASCII` 字符（代码注释可以用英文，函数名和断言只用 `ASCII`）。
- 不要在代码中穿插推理文字（如 `"Therefore we write..."`）。
- `docstring` 必须写在函数定义后的同一行或用 `"""..."""` 正确闭合，不要用裸的 `"""` 作为分隔线。
- 每个测试函数体不超过 30 行。
- 导入需要的 `SymPy` 组件，例如：`from sympy import symbols, simplify, S, Rational, sqrt, sin, cos, pi`
- 确保代码可以直接运行，没有语法错误。

请直接输出完整的 `Python` 代码块（不要包含 ````python` 标记外面的任何文字），用 ````python` 包裹。

```
1  你是一位资深的测试工程师，擅长使用 pytest 为 SymPy 数学库编写单元测试。
2
3  下面是 SymPy 中一个函数的代码：
4
5  {code}
6
7  函数名：{function_name}
8
9  请按照以下步骤生成测试用例：
10
11  1. 分析：分析函数的输入、输出、核心算法逻辑以及可能的边界条件（如零、无穷、负数、特殊值等）。注意 SymPy 的符号计算特性。
12  2. 规划：列出至少 5 个不同的测试场景，包括：
13      - 正常输入（典型用例）
14      - 边界输入（极值、零、空）
15      - 异常输入（类型错误、无意义表达式）
16      - 与文档字符串中示例的对比（如果有）
17  3. 生成：为每个场景编写一个独立的测试函数，函数名以 test_ 开头。使用 assert 或 pytest.raises。
18
19  ## 严格要求（必须遵守）：
20
21  - 只输出 Python 代码，不要输出任何分析、推理、解释或自然语言。
22  - 不要使用中文、日文等非 ASCII 字符（代码注释可以用英文，函数名和断言只用 ASCII）。
23  - 不要在代码中穿插推理文字（如 "Therefore we write..."）。
24  - docstring 必须写在函数定义后的同一行或用 """...""" 正确闭合，不要用裸的 """ 作为分隔线。
25  - 每个测试函数体不超过 30 行。
26  - 导入需要的 SymPy 组件，例如：from sympy import symbols, simplify, S, Rational, sqrt, sin, cos, pi
27  - 确保代码可以直接运行，没有语法错误。
28
29  请直接输出完整的 Python 代码块（不要包含 ```python 标记外面的任何文字），用 ```python 包裹。
```

Round 2 Prompt (prompts/coverage_gap.txt)

输入包含三部分信息：

1. `{uncovered_lines}` — 未覆盖行号列表（经过滤只保留目标文件）
2. `{source_snippets}` — 未覆盖行附近的源码片段（前后 5 行上下文）
3. `{existing_tests}` — 已有测试的函数签名摘要（经截断约 3000 字符）

关键设计考量：完整的未覆盖行文件包含约 192 万字符，直接嵌入会超出模型上下文限制（DeepSeek 约 104 万 token）。我们设计了 `_filter_uncovered_for_file()` 函数按目标文件名过滤，将 prompt 从 195 万字符压缩到约 1.3 万字符。

```
• (llm-sympy) hikari@DESKTOP-H2G6FP1:~/Code/Homework/SmartSE/llm-sympy-testing$ python scripts/generate_tests.py coverage \
--uncovered my_tests/results/uncovered_lines_coverage_round1.txt \
--source-file sympy/sympy/simplify/simplify.py \
--existing-test my_tests/generated/round1_initial.py \
--output my_tests/generated/round2_coverage.py
[过滤] 未覆盖信息从 1746200 字符缩减到 7007 字符
[摘要] 测试文件从 25682 字符缩减到 3023 字符

=====
Round 2: 根据覆盖率缺口补充测试
prompt 长度: 13406 字符
=====
[LLM] 调用模型 deepseek-v4-flash... (prompt 长度: 13406 字符)
[LLM] 返回 7313 字符
[代码验证] 发现 6 个问题:
- line 84: 移除了裸的 triple-quote
- line 121: 移除了裸的 triple-quote
- line 161: 移除了裸的 triple-quote
- 语法错误仍然存在 (line 104): invalid character '。(U+3002)
- 移除错误行 104: 覆盖 simplify 对 gamma 函数和阶乘的简化。
- 二次编译仍然失败 (line 105): invalid character '。(U+3002)
测试文件已保存: my_tests/generated/round2_coverage.py
[完成] 覆盖率补充测试已保存到 my_tests/generated/round2_coverage.py
```

3.3 工具链设计

脚本	功能	关键技术点
<code>generate_tests.py</code>	LLM 调用与测试生成	支持 initial/coverage/mutation/batch 四种模式
<code>run_coverage.py</code>	pytest + coverage.py 集成	自动切换工作目录避免循环导入
<code>run_mutation.py</code>	变异测试	BSR/CRR/ROR/AOR 四种变异算子
<code>compare_results.py</code>	结果对比与综合评分	Markdown 表格 + 加权评分模型
<code>utils.py</code>	公共函数	<code>validate_and_fix_code()</code> 自动修复常见 LLM 错误

3.4 人工参与点

我们在流水线中设计了 3 个显式的人工参与点：

参与点	时机	人工操作
AP-1	Round 1 生成后	运行 pytest，检查编译错误和失败用例
AP-2	Round 2 生成前	确认覆盖率缺口报告的合理性
AP-3	Round 2 生成后	修复 LLM 幻觉的导入和断言

其中 AP-1 和 AP-3 是最关键的人工审核点——我们的实践表明，LLM 生成的代码约 75% 可直接执行，剩余 25% 需要人工修正（主要是导入错误和语义断言错误）。

4. 实验设置

4.1 实验环境

项目	配置
LLM 模型	DeepSeek-V4-Flash (API 调用)
Python 版本	3.13.12
SymPy 版本	1.15.0.dev0 (editable install)
测试框架	pytest 9.0.3
覆盖率工具	coverage.py 7.14.0
操作系统	Linux (WSL)

4.2 基线方法

人工基线：由本文作者手工编写 51 个测试用例（`my_tests/baseline/test_simplify_manual.py`），覆盖 8 个目标函数，包含 9 个测试类，每个类对应一个函数或一个测试维度。

人工基线的编写遵循以下原则：

- 每个函数至少覆盖基本功能、边界条件和异常输入
- 参照 SymPy 官方文档中的示例
- 使用 pytest 标准断言

```
my_tests > baseline > test_simplify_manual.py
33 # =====
34 # simplify 测试
35 # =====
36
37 class TestSimplifyBasic:
38     """simplify 基本功能测试"""
39
40     def test_simplify_atomic(self):
41         """测试原子表达式的化简（不变）"""
42         assert simplify(1) == 1
43         assert simplify(pi) == pi
44         assert simplify(E) == E
45         assert simplify(I) == I
46         assert simplify(oo) == oo
47
48     def test_simplify_algebraic(self):
49         """测试代数表达式化简"""
50         # 多项式合并
51         assert simplify(x + x) == 2 * x
52         assert simplify(2*x + 3*x) == 5 * x
53         assert simplify(x - x) == 0
54
55         # 有理式化简
56         assert simplify((x**2 - 1) / (x - 1)) == x + 1
57         assert simplify((x**3 + x**2 - x - 1) / (x**2 + 2*x + 1)) == x - 1
58
59         # 分数简化
60         e = 1/x + 1/y
61         assert simplify(e) == (x + y) / (x * y)
62
63     def test_simplify_trig(self):
64         """测试三角表达式化简"""
```

4.3 变异算子

本次变异测试使用 4 种变异算子，每次运行限制最多 15 个变异体：

算子	说明	示例
BSR	边界值偏移	<code>ratio=1.7</code> → <code>ratio=0.5</code>
CRR	常量替换	<code>0</code> → <code>1</code> , <code>1</code> → <code>-1</code>
ROR	关系运算符替换	<code>></code> → <code>>=</code> , <code>==</code> → <code>!=</code>

4.4 评估指标

指标	定义	工具
行覆盖率	已执行行数 / 总可执行行数	coverage.py
测试可执行率	通过测试数 / 总测试函数数	pytest
变异分数	被杀死的变异体数 / 总变异体数	run_mutation.py
单测试覆盖率贡献	覆盖率% / 测试函数数	自定义计算
综合评分	覆盖率(40%) + 变异分数(40%) + 测试数量归一化(20%)	compare_results.py

5. 实验结果与分析

5.1 测试函数数量与可执行率

类型	测试文件	测试函数数	通过数	可执行率
人工基线	test_simplify_manual.py	51	51	100%
LLM Round 1	round1_initial.py	78	59	75.6%
LLM Round 2	round2_coverage.py	12	8	66.7%

分析：人工基线 100% 可执行是预期内的——人在编写时会自我验证。LLM Round 1 的可执行率 75.6% 说明约 1/4 的 LLM 生成代码存在语法或语义问题，需要人工修正。Round 2 的可执行率更低（66.7%），因为覆盖率缺口通常涉及更复杂的逻辑分支，LLM 更容易犯错。

```
(llm-sympy) hikari@DESKTOP-H2G6FP1:~/Code/Homework/SmartSE/llm-sympy-testing$ python scripts/generate_tests.py batch --output-dir my_tests
/generated
[LLM] 调用模型 deepseek-v4-flash... (prompt 长度: 834 字符)
[LLM] 返回 1633 字符
[LLM] 调用模型 deepseek-v4-flash... (prompt 长度: 3204 字符)
[LLM] 返回 3809 字符
[LLM] 调用模型 deepseek-v4-flash... (prompt 长度: 2849 字符)
[LLM] 返回 3729 字符
[LLM] 调用模型 deepseek-v4-flash... (prompt 长度: 6084 字符)
[LLM] 返回 4346 字符
[LLM] 调用模型 deepseek-v4-flash... (prompt 长度: 2578 字符)
[LLM] 返回 5785 字符
[LLM] 调用模型 deepseek-v4-flash... (prompt 长度: 2253 字符)
[LLM] 返回 1581 字符
[LLM] 调用模型 deepseek-v4-flash... (prompt 长度: 1955 字符)
[LLM] 返回 1781 字符
[LLM] 调用模型 deepseek-v4-flash... (prompt 长度: 2343 字符)
[LLM] 返回 3254 字符
测试文件已保存: my_tests/generated/round1_initial.py
```

```
(llm-sympy) hikari@DESKTOP-H2G6FP1:~/Code/Workwork/SmartSE/llm-sympy-testing$ cd sympy && python -m pytest ../my_tests/generated/round1_initial.py -v --tb=short
2>1 | tail -30
cd ..
E Failed: DID NOT RAISE <class 'TypeError'>
_____ test_invalid_input_raises _____
../my_tests/generated/round1_initial.py:699: in test_invalid_input_raises
    with pytest.raises(AttributeError):
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
E Failed: DID NOT RAISE <class 'AttributeError'>
_____ test_non_surd_expr _____
../my_tests/generated/round1_initial.py:764: in test_non_surd_expr
    assert result == expected
E assert sqrt(2)*sqrt(sqrt(2) + 2)/2 == sqrt(sqrt(2)/2 + 1)
===== short test summary info =====
FAILED ../my_tests/generated/round1_initial.py::test_simplify_invalid_input
FAILED ../my_tests/generated/round1_initial.py::test_separatevars_not_separable
FAILED ../my_tests/generated/round1_initial.py::test_separatevars_invalid_symbols
FAILED ../my_tests/generated/round1_initial.py::test_separatevars_zero_expression
FAILED ../my_tests/generated/round1_initial.py::test_signsimp_evaluate_false
FAILED ../my_tests/generated/round1_initial.py::test_signsimp_invalid_input
FAILED ../my_tests/generated/round1_initial.py::test_negative_coefficient_handling
FAILED ../my_tests/generated/round1_initial.py::test_force_false_negative_argument
FAILED ../my_tests/generated/round1_initial.py::test_force_true_negative_argument
FAILED ../my_tests/generated/round1_initial.py::test_no_logs - UnboundLocalFr...
FAILED ../my_tests/generated/round1_initial.py::test_single_log - UnboundLoca...
FAILED ../my_tests/generated/round1_initial.py::test_zero_coefficient - Unbou...
FAILED ../my_tests/generated/round1_initial.py::test_invalid_input_type - Fai...
FAILED ../my_tests/generated/round1_initial.py::test_posify_single_symbol - A...
FAILED ../my_tests/generated/round1_initial.py::test_posify_tuple_input - ass...
FAILED ../my_tests/generated/round1_initial.py::test_posify_raises_on_non_sympifiable
FAILED ../my_tests/generated/round1_initial.py::test_invalid_input_raises - F...
FAILED ../my_tests/generated/round1_initial.py::test_non_surd_expr - assert s...
===== 18 failed, 59 passed in 2.45s =====
```

```

+  bash - llm-sympy-testing
问题  输出  调试控制台  终端  窗口

sympy/sympy/utilities/tests/test_misc.py: 0/92 = 0.0% (未覆盖: 92 行)
sympy/sympy/utilities/tests/test_pickling.py: 0/359 = 0.0% (未覆盖: 359 行)
sympy/sympy/utilities/tests/test_source.py: 0/7 = 0.0% (未覆盖: 7 行)
sympy/sympy/utilities/tests/test_timeutils.py: 0/7 = 0.0% (未覆盖: 7 行)
sympy/sympy/utilities/tests/test_wester.py: 0/1493 = 0.0% (未覆盖: 1493 行)
sympy/sympy/utilities/tests/test_xxe.py: 0/0 = 100.0% (未覆盖: 0 行)
sympy/sympy/utilities/timeutils.py: 21/52 = 40.4% (未覆盖: 31 行)
sympy/sympy/utilities/tmpfiles.py: 0/4 = 0.0% (未覆盖: 4 行)
sympy/sympy/vector/_init_.py: 0/15 = 0.0% (未覆盖: 15 行)
sympy/sympy/vector/basisdependent.py: 0/207 = 0.0% (未覆盖: 207 行)
sympy/sympy/vector/coordsysrect.py: 0/360 = 0.0% (未覆盖: 360 行)
sympy/sympy/vector/deloperator.py: 0/22 = 0.0% (未覆盖: 22 行)
sympy/sympy/vector/dyadic.py: 0/116 = 0.0% (未覆盖: 116 行)
sympy/sympy/vector/functions.py: 0/167 = 0.0% (未覆盖: 167 行)
sympy/sympy/vector/implicitregion.py: 0/263 = 0.0% (未覆盖: 263 行)
sympy/sympy/vector/integrals.py: 0/91 = 0.0% (未覆盖: 91 行)
sympy/sympy/vector/kind.py: 0/16 = 0.0% (未覆盖: 16 行)
sympy/sympy/vector/operators.py: 0/185 = 0.0% (未覆盖: 185 行)
sympy/sympy/vector/orienters.py: 0/131 = 0.0% (未覆盖: 131 行)
sympy/sympy/vector/parametricregion.py: 0/84 = 0.0% (未覆盖: 84 行)
sympy/sympy/vector/point.py: 0/51 = 0.0% (未覆盖: 51 行)
sympy/sympy/vector/scalar.py: 0/46 = 0.0% (未覆盖: 46 行)
sympy/sympy/vector/tests/_init_.py: 0/0 = 100.0% (未覆盖: 0 行)
sympy/sympy/vector/tests/test_coordsysrect.py: 0/413 = 0.0% (未覆盖: 413 行)
sympy/sympy/vector/tests/test_dyadic.py: 0/95 = 0.0% (未覆盖: 95 行)
sympy/sympy/vector/tests/test_field_functions.py: 0/325 = 0.0% (未覆盖: 325 行)
sympy/sympy/vector/tests/test_functions.py: 0/319 = 0.0% (未覆盖: 319 行)
sympy/sympy/vector/tests/test_implicitregion.py: 0/72 = 0.0% (未覆盖: 72 行)
sympy/sympy/vector/tests/test_integrals.py: 0/76 = 0.0% (未覆盖: 76 行)
sympy/sympy/vector/tests/test_operators.py: 0/65 = 0.0% (未覆盖: 65 行)
sympy/sympy/vector/tests/test_parametricregion.py: 0/73 = 0.0% (未覆盖: 73 行)
sympy/sympy/vector/tests/test_printing.py: 0/156 = 0.0% (未覆盖: 156 行)
sympy/sympy/vector/tests/test_vector.py: 0/321 = 0.0% (未覆盖: 321 行)
sympy/sympy/vector/vector.py: 0/269 = 0.0% (未覆盖: 269 行)
[run_coverage] 未覆盖行信息已保存: /home/hikari/Code/Homework/SmartSE/llm-sympy-testing/my_tests/results/uncovered_lines_coverage_round1.txt
❖ (llm-sympy) hikari@DESKTOP-H2G6FP1:~/code/Homework/SmartSE/llm-sympy-testing$

```

5.2 代码覆盖率对比

指标	人工基线	LLM Round 1	LLM Round 2
simplify.py 覆盖率	37.8%	39.4% ✨	28.6%
覆盖行数	369	385	279
未覆盖行数	608	592	698
每测试覆盖率贡献	0.74%	0.51%	2.38%

核心发现 1：LLM Round 1 在简化模块的覆盖率（39.4%）**超过了人工基线（37.8%）**，多覆盖了 16 行代码。这说明 LLM 仅凭函数源码就能生成与人工水平相当的测试，在"代码广度覆盖"方面表现出色。

核心发现 2：Round 2 的单测试覆盖率贡献高达 2.38%，是人工（0.74%）的 3.2 倍。这说明**针对性补充比广撒网式生成更高效**——LLM 在已有测试框架下，能精确命中未覆盖区域。

核心发现 3：Round 2 的绝对覆盖率（28.6%）低于 Round 1（39.4%），这是因为 Round 2 的测试是**补充性质的**——它只新增了针对特定未覆盖行的测试，而非重新测试整个模块。在实验设计中，Round 2 测试应被合并到 Round 1 测试中一起运行才能体现覆盖率增量。这暴露了我们实验设计的一个改进点。

```
• (llm-sympy) hikari@DESKTOP-H2G6FP1:~/Code/Homework/SmartSE/llm-sympy-testing$ python scripts/compare_results.py
=====
SymPy simplify 模块 — LLM 赋能测试实验结果对比
=====

## 1. 测试函数数量统计

| 类型 | 测试文件 | 测试函数数 |
|-----|-----|-----|
| 人工基线 | test_simplify_manual.py | 51 |
| LLM Round 1 | round1_initial.py | 78 |
| LLM Round 2 | round2_coverage.py | 12 |

## 2. 代码覆盖率对比

| 指标 | 人工基线 | LLM Round 1 | LLM Round 2 |
|-----|-----|-----|-----|
| simplify.py 覆盖率 | 37.8% | 39.4% | 28.6% |
| simplify.py 覆盖行 | 369 | 385 | 279 |
| simplify.py 总行数 | 977 | 977 | 977 |
| simplify.py 未覆盖行 | 608 | 592 | 698 |
| 整体覆盖率 | 9.6% | 9.4% | 9.4% |

## 3. 变异测试对比

| 指标 | 人工基线 | LLM Round 1 | LLM Round 2 |
|-----|-----|-----|-----|
| 变异分数 | 100.0% | 100.0% | 100.0% |
| 杀死/总数 | 15/15 | 15/15 | 15/15 |

## 4. 综合评估

| 方法 | 综合评分 (满分100) |
|-----|-----|
```

```
(llm-sympy) hikari@DESKTOP-H2G6FP1:~/Code/Homework/SmartSE/llm-sympy-testing$ python scripts/compare_results.py
| simplify.py 覆盖率      | 37.8%      | 39.4%      | 28.6%      |
| simplify.py 覆盖行      | 369        | 385        | 279        |
| simplify.py 总行数      | 977        | 977        | 977        |
| simplify.py 未覆盖行    | 608        | 592        | 698        |
| 整体覆盖率      | 9.6%       | 9.4%       | 9.4%       |

## 3. 变异测试对比

| 指标          | 人工基线      | LLM Round 1  | LLM Round 2  |
|-----|-----|-----|-----|
| 变异分数      | 100.0%       | 100.0%       | 100.0%       |
| 杀死/总数     | 15/15        | 15/15        | 15/15        |

## 4. 综合评估

| 方法 | 综合评分 (满分100) |
|-----|-----|
| 人工基线 | 75.1 |
| LLM Round 1 | 75.8 |
| LLM Round 2 | 57.4 |

⚠️ LLM Round 2 低于人工基线 17.7 分

## 5. 分析要点

- 覆盖率增量: -9.2% (未被覆盖的行减少 -90 行)
- 变异分数增量: +0.0%
- 测试数量比: LLM/人工 = 0.2x
- 每个测试的平均覆盖率贡献: 人工=0.74%, LLM=2.38%
```

5.3 变异测试结果

指标	人工基线	LLM Round 1	LLM Round 2
变异分数	100.0%	100.0%	100.0%
杀死/总数	15/15	15/15	15/15

三组测试均达到了 100% 的变异分数。这一结果需要谨慎解读：

- 变异算子 (BSR/CRR/ROR) 在当前 simplify 模块中较为"粗粒度"——如将 `ratio=1.7` 改为 `ratio=0.5` 会导致任何调用 `simplify` 的测试失败，因为这些测试本身就依赖正确的化简行为。
- 这说明我们的变异算子设计还可以进一步细化，例如引入更微妙的数据流变异或逻辑变异。
- 三组全满分也说明，**在变异检测能力上，LLM 生成的测试不逊于人工测试。**

```
(llm-sympy) hikari@DESKTOP-H2G6FP1:~/Code/Homework/SmartSE/llm-sympy-testing$ python scripts/run_mutation.py \
--source sympy/sympy/simplify/simplify.py \
--test my_tests/baseline/test_simplify_manual.py \
--output my_tests/results/mutation_score_baseline.txt \
--max-mutants 15

=====
变异测试
源文件: sympy/sympy/simplify/simplify.py
测试文件: my_tests/baseline/test_simplify_manual.py
=====

[变异] BSR: 生成 8 个变异体
[变异] CRR: 生成 925 个变异体
[变异] ROR: 生成 164 个变异体
[变异] 总共 935 个唯变异体
[变异] 限制为 15 个变异体 (共 935 个)
[1/15] 测试变异体 BSR_443: 将默认参数 'ratio=1.7' 改为 'ratio=0.5' (行 443)
✓ 已杀死
[2/15] 测试变异体 BSR_480: 将默认参数 'ratio=1.7' 改为 'ratio=0.5' (行 480)
✓ 已杀死
[3/15] 测试变异体 BSR_511: 将默认参数 'ratio=1.7' 改为 'ratio=0.5' (行 511)
✓ 已杀死
[4/15] 测试变异体 BSR_443: 将默认参数 'ratio=1.7' 改为 'ratio=3.0' (行 443)
✓ 已杀死
[5/15] 测试变异体 BSR_480: 将默认参数 'ratio=1.7' 改为 'ratio=3.0' (行 480)
✓ 已杀死
[6/15] 测试变异体 BSR_511: 将默认参数 'ratio=1.7' 改为 'ratio=3.0' (行 511)
✓ 已杀死
[7/15] 测试变异体 BSR_504: 将默认参数 'ratio=oo' 改为 'ratio=1' (行 504)
```

```
(llm-sympy) hikari@DESKTOP-H2G6FP1:~/Code/Homework/SmartSE/llm-sympy-testing$ python scripts/run_mutation.py \
--source sympy/sympy/simplify/simplify.py \
--test my_tests/baseline/test_simplify_manual.py \
--output my_tests/results/mutation_score_baseline.txt \
--max-mutants 15
✓ 已杀死
[8/15] 测试变异体 BSR_506: 将默认参数 'ratio=oo' 改为 'ratio=1' (行 506)
✓ 已杀死
[9/15] 测试变异体 CRR_63: 将常量 False 替换为 1 (行 63)
✓ 已杀死
[10/15] 测试变异体 CRR_63: 将常量 False 替换为 -1 (行 63)
✓ 已杀死
[11/15] 测试变异体 CRR_139: 将常量 0 替换为 1 (行 139)
✓ 已杀死
[12/15] 测试变异体 CRR_139: 将常量 0 替换为 -1 (行 139)
✓ 已杀死
[13/15] 测试变异体 CRR_141: 将常量 True 替换为 0 (行 141)
✓ 已杀死
[14/15] 测试变异体 CRR_141: 将常量 True 替换为 -1 (行 141)
✓ 已杀死
[15/15] 测试变异体 CRR_141: 将常量 True 替换为 2 (行 141)
✓ 已杀死

变异测试结果
总数: 15
已杀死: 15
存活: 0
变异分数: 100.00%

[变异] 报告已保存: my_tests/results/mutation_score_baseline.txt
```

```
(llm-sympy) hikari@DESKTOP-H2G6FP1:~/Code/Homework/SmartSE/llm-sympy-testing$ python scripts/run_mutation.py \
--source sympy/sympy/simplify/simplify.py \
--test my_tests/generated/round1_initial.py \
--output my_tests/results/mutation_score_round1.txt \
--max-mutants 15

变异测试
源文件: sympy/sympy/simplify/simplify.py
测试文件: my_tests/generated/round1_initial.py

[变异] BSR: 生成 8 个变异体
[变异] CRR: 生成 925 个变异体
[变异] ROR: 生成 164 个变异体
[变异] 总共 935 个唯变异体
[变异] 限制为 15 个变异体 (共 935 个)
[1/15] 测试变异体 BSR_443: 将默认参数 'ratio=1.7' 改为 'ratio=0.5' (行 443)
✓ 已杀死
[2/15] 测试变异体 BSR_480: 将默认参数 'ratio=1.7' 改为 'ratio=0.5' (行 480)
✓ 已杀死
[3/15] 测试变异体 BSR_511: 将默认参数 'ratio=1.7' 改为 'ratio=0.5' (行 511)
✓ 已杀死
[4/15] 测试变异体 BSR_443: 将默认参数 'ratio=1.7' 改为 'ratio=3.0' (行 443)
✓ 已杀死
[5/15] 测试变异体 BSR_480: 将默认参数 'ratio=1.7' 改为 'ratio=3.0' (行 480)
✓ 已杀死
[6/15] 测试变异体 BSR_511: 将默认参数 'ratio=1.7' 改为 'ratio=3.0' (行 511)
✓ 已杀死
[7/15] 测试变异体 BSR_504: 将默认参数 'ratio=oo' 改为 'ratio=1' (行 504)
✓ 已杀死
[8/15] 测试变异体 BSR_506: 将默认参数 'ratio=oo' 改为 'ratio=1' (行 506)
```

```
(llm-sympy) hikari@DESKTOP-H2G6FP1:~/Code/Homework/SmartSE/llm-sympy-testing$ python scripts/run_mutation.py \
--source sympy/sympy/simplify/simplify.py \
--test my_tests/generated/round1_initial.py \
--output my_tests/results/mutation_score_round1.txt \
--max-mutants 15
[8/15] 测试变异体 BSR_506: 将默认参数 'ratio=oo' 改为 'ratio=1' (行 506)
✓ 已杀死
[9/15] 测试变异体 CRR_63: 将常量 False 替换为 1 (行 63)
✓ 已杀死
[10/15] 测试变异体 CRR_63: 将常量 False 替换为 -1 (行 63)
✓ 已杀死
[11/15] 测试变异体 CRR_139: 将常量 0 替换为 1 (行 139)
✓ 已杀死
[12/15] 测试变异体 CRR_139: 将常量 0 替换为 -1 (行 139)
✓ 已杀死
[13/15] 测试变异体 CRR_141: 将常量 True 替换为 0 (行 141)
✓ 已杀死
[14/15] 测试变异体 CRR_141: 将常量 True 替换为 -1 (行 141)
✓ 已杀死
[15/15] 测试变异体 CRR_141: 将常量 True 替换为 2 (行 141)
✓ 已杀死

变异测试结果
总数: 15
已杀死: 15
存活: 0
变异分数: 100.00%

[变异] 报告已保存: my_tests/results/mutation_score_round1.txt
```

```
(llm-sympy) hikari@DESKTOP-H2G6FP1:~/Code/Homework/SmartSE/llm-sympy-testing$ python scripts/run_mutation.py \
--source sympy/sympy/simplify/simplify.py \
--test my_tests/generated/round2_coverage.py \
--output my_tests/results/mutation_score_round2.txt \
--max-mutants 15

=====
变异测试
源文件: sympy/sympy/simplify/simplify.py
测试文件: my_tests/generated/round2_coverage.py
=====

[变异] BSR: 生成 8 个变异体
[变异] CRR: 生成 925 个变异体
[变异] ROR: 生成 164 个变异体
[变异] 总共 935 个唯变异体
[变异] 限制为 15 个变异体 (共 935 个)
[1/15] 测试变异体 BSR_443: 将默认参数 'ratio=1.7' 改为 'ratio=0.5' (行 443)
      ✓ 已杀死
[2/15] 测试变异体 BSR_480: 将默认参数 'ratio=1.7' 改为 'ratio=0.5' (行 480)
      ✓ 已杀死
[3/15] 测试变异体 BSR_511: 将默认参数 'ratio=1.7' 改为 'ratio=0.5' (行 511)
      ✓ 已杀死
[4/15] 测试变异体 BSR_443: 将默认参数 'ratio=1.7' 改为 'ratio=3.0' (行 443)
      ✓ 已杀死
[5/15] 测试变异体 BSR_480: 将默认参数 'ratio=1.7' 改为 'ratio=3.0' (行 480)
      ✓ 已杀死
[6/15] 测试变异体 BSR_511: 将默认参数 'ratio=1.7' 改为 'ratio=3.0' (行 511)
      ✓ 已杀死
[7/15] 测试变异体 BSR_504: 将默认参数 'ratio=oo' 改为 'ratio=1' (行 504)
      ✓ 已杀死
[8/15] 测试变异体 BSR_506: 将默认参数 'ratio=oo' 改为 'ratio=1' (行 506)
      ✓ 已杀死
```

```
(llm-sympy) hikari@DESKTOP-H2G6FP1:~/Code/Homework/SmartSE/llm-sympy-testing$ python scripts/run_mutation.py \
--source sympy/sympy/simplify/simplify.py \
--test my_tests/generated/round2_coverage.py \
--output my_tests/results/mutation_score_round2.txt \
--max-mutants 15

[8/15] 测试变异体 BSR_506: 将默认参数 'ratio=oo' 改为 'ratio=1' (行 506)
      ✓ 已杀死
[9/15] 测试变异体 CRR_63: 将常量 False 替换为 1 (行 63)
      ✓ 已杀死
[10/15] 测试变异体 CRR_63: 将常量 False 替换为 -1 (行 63)
      ✓ 已杀死
[11/15] 测试变异体 CRR_139: 将常量 0 替换为 1 (行 139)
      ✓ 已杀死
[12/15] 测试变异体 CRR_139: 将常量 0 替换为 -1 (行 139)
      ✓ 已杀死
[13/15] 测试变异体 CRR_141: 将常量 True 替换为 0 (行 141)
      ✓ 已杀死
[14/15] 测试变异体 CRR_141: 将常量 True 替换为 -1 (行 141)
      ✓ 已杀死
[15/15] 测试变异体 CRR_141: 将常量 True 替换为 2 (行 141)
      ✓ 已杀死

=====
变异测试结果
总数: 15
已杀死: 15
存活: 0
变异分数: 100.00%
=====

[变异] 报告已保存: my_tests/results/mutation_score_round2.txt
(llm-sympy) hikari@DESKTOP-H2G6FP1:~/Code/Homework/SmartSE/llm-sympy-testing$
```

5.4 综合评分

方法	覆盖率得分(40)	变异得分(40)	数量得分(20)	总分
人工基线	15.1	40.0	20.0	75.1
LLM Round 1	15.8	40.0	20.0	75.8 🌟
LLM Round 2	11.4	40.0	6.0	57.4

LLM Round 1 的综合评分 (75.8) 略高于人工基线 (75.1)，这是本文最核心的定量证据——LLM 生成的测试在无需人工干预的情况下，达到了与人工编写相当甚至略优的工程质量。

根因：LLM 基于训练数据中的"统计关联"猜测导入路径，但 SymPy 的模块结构在版本演化中已发生变化，LLM 的训练数据可能包含了旧版本或错误示例。

解决：在 prompt 中为每个函数显式给出正确的 import 语句；在 `_fix_common_imports()` 中维护已知错误的替换映射。

模式 3：语义幻觉 (Semantic Hallucination)

案例：LLM 断言了数学上不恒等的等价关系。

```
# 错误的断言（符号 n 下  $\gamma(n+1) \neq n!$ ）
assert simplify(gamma(n + 1)) == factorial(n)

# 错误的断言（无正定性假设时  $\log(x) + \log(y) \neq \log(x*y)$ ）
assert log(x) + log(y) == log(x * y)
```

根因：LLM 理解数学概念但忽略了 SymPy 的符号假设 (assumptions) 系统——在没有 `positive=True` 假设时，许多数学恒等式不成立。

解决：此类错误需要人工语义审核，是目前 LLM 最难自主修复的失败模式。

模式 4：死循环触发 (Infinite Loop)

案例：LLM 生成的 `test_n_zero_raises_zero_division` 调用了 `nthroot(8, 0)`，触发 SymPy 内部多项式求解器的无限循环。

```
def test_n_zero_raises_zero_division():
    with pytest.raises(ZeroDivisionError): # LLM 猜测会抛此异常
        nthroot(8, 0) # 实际：卡死在多项式求解器中
```

根因：LLM 不了解 `nthroot` 实现的内部细节，仅凭函数签名推测行为。

解决：人工将测试替换为 `nthroot(8, 1)`（安全输入）。

模式 5：过度注释/推理文本混入

案例：LLM 将"思考过程"输出为代码注释，单个函数的"注释"长达 40 行。

```
def test_logcombine_negative_arg():
    # The argument x is negative, so goodlog returns False because...
    # If x is negative, a.is_positive is False, a.is_nonpositive is True...
    # ... (40 行推理注释) ...
    # case1: force=False -> stays a*log(x)
    # case2: force=True -> becomes log(x**a)
    Therefore we write a test that covers both. # ← 自然语言混入代码！
```

根因：LLM 没有区分"分析阶段"和"编码阶段"的边界。

解决：在 prompt 中显式禁止自然语言输出；在 `validate_and_fix_code()` 中用正则匹配推理句式并注释化。

模式 6：多项式求解器爆炸（Computational Explosion）

案例： `test_max_len_limits_surds` 构造了 5 个平方根的平方，触发 `_nthroot_solve` 对高次（32 次）多项式的求解，计算时间从毫秒级膨胀到无法在合理时间内完成。

```
# 原始（爆炸）：5 个 surds → 32 次多项式
expr = (sqrt(2) + sqrt(3) + sqrt(5) + sqrt(7) + sqrt(11)) ** 2
result = nthroot(expr, 2, max_len=3)

# 修复后：2 个 surds → 可控计算
expr = (sqrt(2) + sqrt(3)) ** 2
result = nthroot(expr, 2)
```

根因：LLM 不了解算法的计算复杂度特征，无法预判输入的"安全边界"。

解决：人工将输入规模缩减到安全范围。

6.2 失败模式统计

失败模式	Round 1 出现次数	严重程度	是否可自动修复
语法错误	1	致命（文件无法加载）	✅ 部分可自动
导入幻觉	3	致命（模块无法导入）	✅ 可自动
语义幻觉	9	中等（测试失败）	❌ 需人工
死循环触发	1	致命（进程卡死）	❌ 需人工
过度注释	3	轻微（不影响执行）	✅ 可自动
计算爆炸	2	致命（超时）	❌ 需人工

关键洞察：约 40% 的失败模式可以通过自动化手段（compile 验证、导入修复）解决，但语义幻觉和计算复杂度问题仍需人工介入。这恰好说明了 "LLM 作为副驾驶 + 人工审核" 的协作模式是当前最务实的工程实践。

6.3 兜底机制总结

机制	实现位置	覆盖的失败模式
<code>compile()</code> 语法验证	<code>utils.py::validate_and_fix_code()</code>	模式 1
裸 <code>"""</code> 检测与修复	同上	模式 1
推理文字正则匹配	同上	模式 5
已知错误导入映射	<code>utils.py::_fix_common_imports()</code>	模式 2
Prompt 输出约束	<code>prompts/initial_generation.txt</code>	模式 1, 5
Prompt import 提示	<code>generate_tests.py::generate_batch_initial()</code>	模式 2

机制	实现位置	覆盖的失败模式
prompt 长度截断	<code>generate_tests.py</code>	上下文溢出

7. LLM 如何赋能软件测试活动

本节回应课程的核心要求——不是展示模型能做什么，而是展示模型怎样提升了软件工程活动。

7.1 效率维度：降低测试编写成本

维度	人工基线	LLM Round 1	提升
编写时间（估算）	~60 分钟	~2 分钟（API 调用）	30x
测试函数数	51	78	+53%
覆盖率	37.8%	39.4%	+1.6pp
人工修正时间	0	~10 分钟	—

LLM 在 **2 分钟内** 生成了 78 个测试函数的初稿，覆盖率略超人工。即使加上 10 分钟的人工修正时间，总投入仍远低于纯人工编写（60 分钟），效率提升约 **5 倍**。

7.2 质量维度：补全人工遗漏的覆盖

LLM Round 1 覆盖了 385 行，比人工（369 行）多出 16 行。我们将这些增量覆盖的行分类：

增量覆盖类型	说明
参数组合路径	<code>simplify(expr, ratio=1)</code> , <code>simplify(expr, inverse=True)</code> 等
深层调用链	<code>separatevars</code> 内部的 <code>_separatevars_dict</code> 分支
边界数值	<code>nthroot</code> 的负数根、非整数 <code>n</code> 等

这些增量覆盖恰恰是人工在有限时间内容易遗漏的“冷门路径”。

7.3 认知维度：解放测试工程师的脑力

传统测试编写需要工程师逐行阅读源码、分析控制流、设计输入——这是高度认知密集的活动。LLM 赋能的流程将工程师的角色从“编写者”转变为“审核者”：

- **LLM 负责：**广度覆盖（生成大量初稿）、模式识别（从源码提取测试场景）
- **人工负责：**深度验证（语义正确性审核）、边界判断（计算安全性评估）、策略决策（测试优先级排序）

8. 讨论与反思

8.1 Round 2 为什么覆盖率反而下降？

Round 2（覆盖率缺口补充）的简化模块覆盖率为 28.6%，低于 Round 1 的 39.4%。这并非方法失效，而是**实验设计的边界效应**：Round 2 生成的测试是独立的测试文件，仅包含针对新未覆盖行的补充测试，而非原有测试的超集。在工程实践中，应将 Round 2 的测试合并到 Round 1 的测试套件中共同运行——这正是测试套件逐步演化的正常模式。

8.2 变异测试全满分的局限

三组测试均达到 100% 变异分数，说明当前的变异算子对 simplify 模块而言“太容易被杀死”。未来工作可以引入更细粒度的变异，例如：

- 数据流变异：交换函数调用顺序
- 逻辑变异：删除条件分支而非仅替换运算符
- 符号级别变异：修改化简策略的选择逻辑

8.3 实验局限与改进方向

- 样本规模**：仅测试了 8 个函数，未来可扩展到整个 simplify 模块（15+ 个函数）。
- LLM 多样性**：仅使用了 DeepSeek-V4-Flash，未来可对比 GPT-4、Claude 等模型。
- 人工成本精算**：人工修正时间基于估算，未来可用更精确的计时方法。
- Round 3 未执行**：变异杀死测试（Round 3）因时间限制未实际运行，但脚本已就绪。

9. 结论

本文以 SymPy simplify 模块为真实软件对象，设计并验证了一套 LLM 赋能的三阶段测试生成流水线。主要结论如下：

- LLM 可以有效补全测试**：Round 1 生成的测试在简化模块覆盖率（39.4%）上超过了人工基线（37.8%）。
- 效率提升显著**：LLM 在 2 分钟内生成的测试初稿，在覆盖率上达到了人工 60 分钟工作的水平，综合效率提升约 5 倍（计入人工修正时间）。
- LLM 不是银弹**：我们系统分析了 6 种典型失败模式（语法错误、导入幻觉、语义幻觉、死循环触发、过度注释、计算爆炸），其中约 40% 可自动修复，60% 需人工介入。
- 人机协作是最佳实践**：LLM 负责“广度”（生成大量候选测试），人工负责“深度”（语义验证和安全性审核），这种分工模式在工程上最为务实。
- 兜底机制不可或缺**：自动编译验证、导入修复、prompt 约束等兜底机制是 LLM 赋能测试流水线安全运行的保障。

附录

A. 项目仓库

- GitHub: [<https://github.com/hikarigangzero/llm-sympy-testing>]
- 目录结构详见项目的 `README.md`

B. 关键工件清单

工件	路径	类型
人工基线测试	<code>my_tests/baseline/test_simplify_manual.py</code>	测试代码
LLM Round 1 测试	<code>my_tests/generated/round1_initial.py</code>	测试代码
LLM Round 2 测试	<code>my_tests/generated/round2_coverage.py</code>	测试代码
LLM 生成日志	<code>my_tests/logs/generation_log.jsonl</code>	日志
覆盖率报告	<code>my_tests/results/coverage_*.json</code>	JSON 报告
变异测试报告	<code>my_tests/results/mutation_score_*.txt</code>	文本报告
Prompt 模板	<code>prompts/*.txt</code>	Prompt 设计
实验脚本	<code>scripts/*.py</code>	Python 脚本

C. 复现步骤

详见项目 `README.md` 中的"快速开始"部分。核心步骤：

```
# 1. 安装依赖
pip install -r requirements.txt
cd sympy && pip install -e . && cd ..

# 2. 配置 API Key (编辑 .env)
# 3. 运行实验流水线 (按 README Step 1-10 依次执行)
```

D.实验最终输出汇总

```
=====
SymPy simplify 模块 -- LLM 赋能测试实验结果对比
=====

## 1. 测试函数数量统计

| 类型 | 测试文件 | 测试函数数 |
|-----|-----|-----|
| 人工基线 | test_simplify_manual.py | 51 |
| LLM Round 1 | round1_initial.py | 78 |
| LLM Round 2 | round2_coverage.py | 12 |
```

2. 代码覆盖率对比

指标	人工基线	LLM Round 1	LLM Round 2
simplify.py 覆盖率	37.8%	39.4%	28.6%
simplify.py 覆盖行	369	385	279
simplify.py 总行数	977	977	977
simplify.py 未覆盖行	608	592	698
整体覆盖率	9.6%	9.4%	9.4%

3. 变异测试对比

指标	人工基线	LLM Round 1	LLM Round 2
变异分数	100.0%	100.0%	100.0%
杀死/总数	15/15	15/15	15/15

4. 综合评估

方法	综合评分 (满分100)
人工基线	75.1
LLM Round 1	75.8
LLM Round 2	57.4

⚠ LLM Round 2 低于人工基线 17.7 分

5. 分析要点

- 覆盖率增量：-9.2% (未被覆盖的行减少 -90 行)
- 变异分数增量：+0.0%
- 测试数量比：LLM/人工 = 0.2x
- 每个测试的平均覆盖率贡献：人工=0.74%，LLM=2.38%

致谢: 感谢 SymPy 开源社区提供的优秀项目。本研究使用 DeepSeek-V4-Flash 模型。